

March 09<sup>th</sup>, 2024

# Interrupts Assignment Report

Embedded System 3, LAB 02

*Document Version 0.0*

**Edited by:**

Farros Ramzy (3767353)

## **Abstract**

This technical report describes about the external interrupt registers in the STM32 board to switch task of a program while it is still doing its main program. The procedure, context, and knowledge that is written in this document is meant to explain how to setup the pin registers of the STM32-F303RE board and the external interrupt to recognize the interrupt line between each pin in GPIO port.

## List of Figures

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Figure 2. 1. 1 External Interrupt (EXTI) pin line. ....                          | 2  |
| Figure 2. 1. 2 RCC APB2ENR Register map.....                                     | 3  |
| Figure 2. 1. 3 SYSCFGEN mode register. ....                                      | 3  |
| Figure 2. 1. 4 EXTICR1 or EXTICR[0] register map.....                            | 3  |
| Figure 2. 1. 5 Basic interrupt wiring diagram. (two buttons and two LEDs).....   | 4  |
|                                                                                  |    |
| Figure 2. 2. 1 RCC GPIO register enabler map. ....                               | 5  |
| Figure 2. 2. 2 GPIO mode register map.....                                       | 6  |
| Figure 2. 2. 3 GPIO output type register map.....                                | 6  |
| Figure 2. 2. 4 GPIO speed register map. ....                                     | 7  |
|                                                                                  |    |
| Figure 2. 3. 1 A large delay to test out the interrupt handler. ....             | 8  |
|                                                                                  |    |
| Figure 2. 4. 1 Extended wiring for polling interrupt experiment.....             | 9  |
| Figure 2. 4. 2 Extended output GPIO. ....                                        | 10 |
| Figure 2. 4. 3 Polling method to test the interrupt experiment. ....             | 11 |
| Figure 2. 4. 4 Enumeration of sweep states. ....                                 | 11 |
|                                                                                  |    |
| Figure 3. 1. 1 Basic interrupt state machine. ....                               | 12 |
|                                                                                  |    |
| Figure 3. 2. 1 Interrupt experiment state machine. ....                          | 13 |
|                                                                                  |    |
| Figure 4. 1. 1 Logic analyzer capture on button click interrupt. ....            | 14 |
| Figure 4. 1. 2 Logic analyzer capture on switch of LED A.....                    | 14 |
| Figure 4. 1. 3 Logic analyzer capture on interrupt button.....                   | 15 |
| Figure 4. 1. 4 Serial print result from the proof of concept manual testing..... | 15 |
|                                                                                  |    |
| Figure 4. 2. 1 Sweep A polling result.....                                       | 16 |
| Figure 4. 2. 2 Logic analyzer result on sweep b. ....                            | 17 |
| Figure 4. 2. 3 Logic analyzer capture on the sweep switch by interrupts. ....    | 17 |
| Figure 4. 2. 4 Manual testing result on serial print.....                        | 18 |

# Table of Contents

|                                                           |    |
|-----------------------------------------------------------|----|
| 1. Introduction .....                                     | 1  |
| 1.1. Equipment .....                                      | 1  |
| 2. Procedure.....                                         | 2  |
| 2.1. Preparation.....                                     | 2  |
| 2.2. Setting up Input and Output Pins from the GPIO ..... | 5  |
| 2.3. Setting up the GPIO EXTI .....                       | 7  |
| 2.4. Experiment (Polling Interrupt) .....                 | 8  |
| 3. Design Concept .....                                   | 12 |
| 3.1. Basic Interrupts .....                               | 12 |
| 3.2. Polling Interrupts.....                              | 13 |
| 4. Results.....                                           | 13 |
| 4.1. Button toggle result .....                           | 14 |
| 4.2. Experiment result .....                              | 16 |
| 5. Conclusion .....                                       | 18 |
| 6. Recommendation.....                                    | 18 |
| 7. References .....                                       | 19 |

# 1. Introduction

This practicum is a learning method to get to know more about the STM32 board, especially on the interrupt registers which is embedded inside it that can be usable for the industrial project and practices in the future.

The requirement of this practicum is to bit-manipulate the STM32 board's interrupt registers to read at least an input in the middle of a task and write at least an output as its interrupt reading result. This method only allowed to use the CMSIS method of coding (hard coded macros and functions required to be made personally). However, "HAL" library is still allowed to use for the GPIO pin and the internal timer ticker of the STM32 board.

Other than that, the assignment is to make sure that the doer of this practicum understands how to use the external interrupt of this STM32 board and be creative to use it on the upcoming projects in the future.

There are three things that must be done for this practicum. The first thing is to investigate on how to use the interrupt registers of the STM32, followed by the design concept of the implementation, and then make at least one proof of concept in practicum.

The proof of concept can be made by some debugs or logical analysis from the implementation results.

Every required equipment is listed in this Equipment section below. And every reference that was being used for this practicum are listed along with the external references in the References section, by the end of this document.

## 1.1. Equipment

The equipment needed to do this research practicum are:

1. A programming platform for the STM32 board (STM32CubeIDE)
2. A designing platform for the implementation (state machines & wiring diagrams)
3. An STM32-F303RE board
4. At least two LEDs (can add more for the experiment purpose)
5. Two pushbuttons
6. A breadboard
7. A bunch of wires
8. A Logic Analyzer
9. One 220  $\Omega$ ,  $\pm 5\%$  resistor for each LED
10. One 1000  $\Omega$ ,  $\pm 5\%$  resistors for each button

## 2. Procedure

### 2.1. Preparation

This experiment is best started when the participants know how the GPIO registers in the STM32 board are working. But to start the assignment, it is required to read the reference manual of the STM32-F303RE board which is listed in this References section below again, especially on the external interrupt register chapter of that reference manual.

In that chapter, should be known that each pin of the STM32 Nucleo GPIO on each port is connected differently on each external interrupt line.

And some lines are grouped as one just like EXTI\_5 to EXTI\_9 on the implementation.

These lines are presented as in this Figure 2. 1. 1 below.

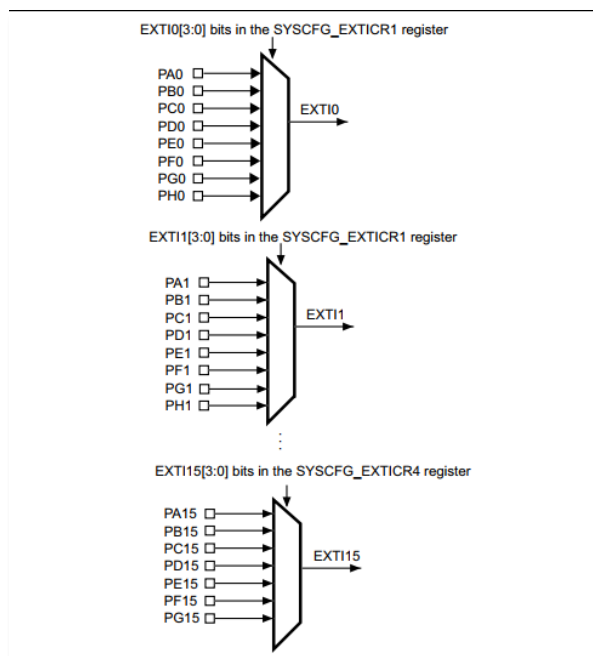


Figure 2. 1. 1 External Interrupt (EXTI) pin line.

From this Figure 2. 1. 1, it is visible that each EXTI line connected to the same pin number in each port of the Nucleo.

Should be noted, there are about 35 lines which actually placed in this EXTI register. However, only line 0 to line 15 which are available to be setup manually for both GPIO input and output.

The rest of them have special purpose on the Nucleo board such as line 23 & 24 which is only available for the I2C wakeup pin for the micro controller board.

To start to use the EXTI or the external interrupt, the SYSCFG register must be activated first. This is because the EXTI configuration (EXTICR) addresses are placed within this SYSCFG register.

This SYSCFG register can be found in the APB2ENR of the RCC register which is shown in this Figure 2. 1. 2 below.

|         |           |         |         |         |     |     |     |     |     |     |          |     |          |          |           |
|---------|-----------|---------|---------|---------|-----|-----|-----|-----|-----|-----|----------|-----|----------|----------|-----------|
| 31      | 30        | 29      | 28      | 27      | 26  | 25  | 24  | 23  | 22  | 21  | 20       | 19  | 18       | 17       | 16        |
| Res     | Res       | Res     | Res     | Res     | Res | Res | Res | Res | Res | Res | TIM20 EN | Res | TIM17 EN | TIM16 EN | TIM15 EN  |
|         |           |         |         |         |     |     |     |     |     |     | rw       |     | rw       | rw       | rw        |
| 15      | 14        | 13      | 12      | 11      | 10  | 9   | 8   | 7   | 6   | 5   | 4        | 3   | 2        | 1        | 0         |
| SPI4E N | USART 1EN | TIM8 EN | SPI1 EN | TIM1 EN | Res | Res | Res | Res | Res | Res | Res      | Res | Res      | Res      | SYS CFGEN |
| rw      | rw        | rw      | rw      | rw      |     |     |     |     |     |     |          |     |          |          | rw        |

Figure 2. 1. 2 RCC APB2ENR Register map.

From that Figure 2. 1. 2 above, it can be known that the SYSCFG register is placed on pin zero of the APB2ENR register. Other than that, it also visible that the register is only use one pin setup.

In this Figure 2. 1. 3 below, it can be known that the SYSCFG register only need 0 or1 input value to change its state.

Bits 10:1 Reserved, must be kept at reset value.

Bit 0 **SYSCFGEN**: SYSCFG clock enable  
Set and cleared by software.  
0: SYSCFG clock disabled  
1: SYSCFG clock enabled

Figure 2. 1. 3 SYSCFGEN mode register.

After setting up this register to one or high, the EXTI pin can be activated.

To setup the EXTI pin, EXTICR line must be called from the SYSCFG register. In this example from Figure 2. 1. 4 below, the EXTICR1 or also known as EXTICR[0] in the coding platform of STM32 Cube IDE, contains a couple of EXTI lines which are from EXTI0 to EXTI3.

|            |      |      |      |            |      |      |      |            |      |      |      |            |      |      |      |
|------------|------|------|------|------------|------|------|------|------------|------|------|------|------------|------|------|------|
| 31         | 30   | 29   | 28   | 27         | 26   | 25   | 24   | 23         | 22   | 21   | 20   | 19         | 18   | 17   | 16   |
| Res.       | Res. | Res. | Res. | Res.       | Res. | Res. | Res. | Res.       | Res. | Res. | Res. | Res.       | Res. | Res. | Res. |
| 15         | 14   | 13   | 12   | 11         | 10   | 9    | 8    | 7          | 6    | 5    | 4    | 3          | 2    | 1    | 0    |
| EXTI3[3:0] |      |      |      | EXTI2[3:0] |      |      |      | EXTI1[3:0] |      |      |      | EXTI0[3:0] |      |      |      |
| rw         | rw   | rw   | rw   | rw         | rw   | rw   | rw   | rw         | rw   | rw   | rw   | rw         | rw   | rw   | rw   |

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:12 **EXTI3[3:0]**: EXTI 3 configuration bits

These bits are written by software to select the source input for the EXTI3 external interrupt.

x000: PA[3] pin  
x001: PB[3] pin  
x010: PC[3] pin  
x011: PD[3] pin  
x100: PE[3] pin  
x101: PF[3] pin  
x110: PG[3] pin  
other configurations: reserved

Figure 2. 1. 4 EXTICR1 or EXTICR[0] register map.

And each EXTI in that Figure 2. 1. 4 also contains pin line which is connected directly to the GPIO pin of the Nucleo board which are written as PA to PG. These follows from pin 0 to 15, which are placed on EXTICR[0] to EXTICR[3].

After done configuring the external interrupt, preparing the interrupt flag is needed.

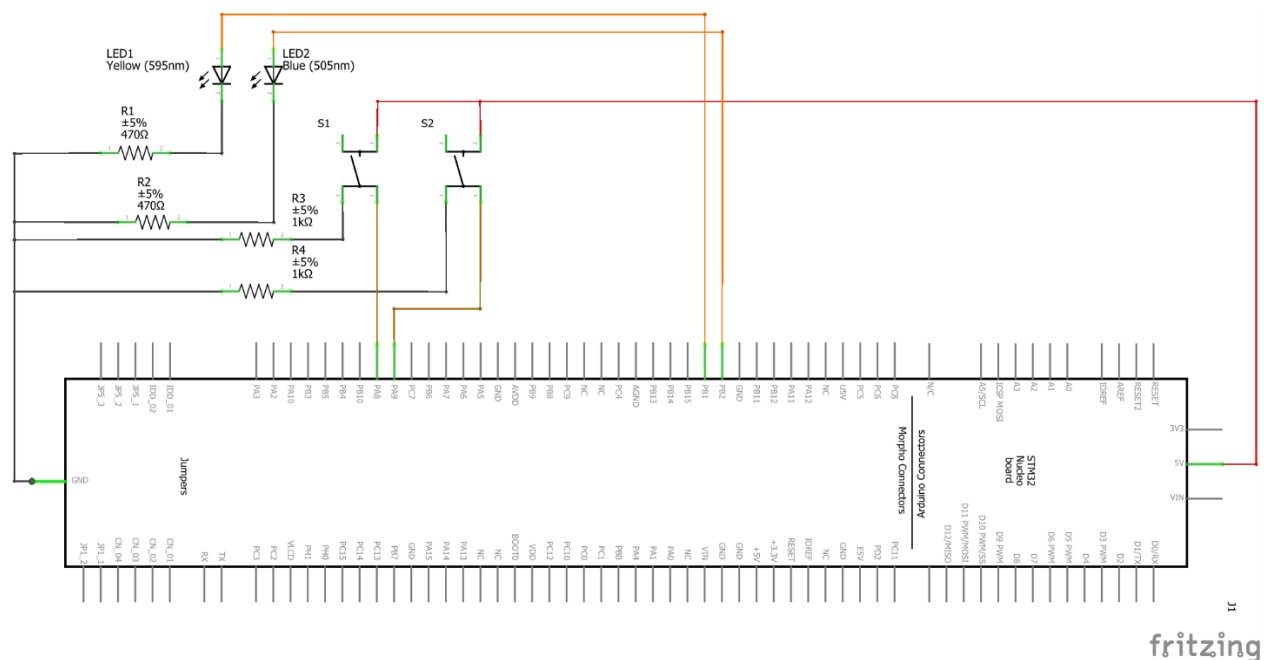
This flag is working like a trigger to detect where exactly the interrupt is being called to the program during the running loop process.

There are two types of interrupt flags. The first one is the FTSR which is the falling trigger, and the second one is the RTSR which is the rising trigger.

Falling trigger is used when the system needs to read a trigger external pin that turned from high to low. And the Rising trigger is used when the system needs to read the opposite change from the external trigger pin.

These triggers than can be handled in the EXTI IRQ handler function which is predefined in the STM32 program to run the interrupt task. However, the logic inside this handler function must be determined by the code maintainers themselves.

To do the proof of concept for these reference manual instructions, a system built by the tools which are listed in the Equipment above can be prepared, following to this Figure 2. 1. 5 below.



Once the wiring is finished, now the source code of the system can be created, following to the steps which are listed below. And after that, an implementation that follows the concept design in the Basic Interrupts chapter can be created.

## 2.2. Setting up Input and Output Pins from the GPIO

To start on the setup, GPIO pins must be configured first. There are two input and output pins that must be configured in total, based on the Figure 2. 1. 5 above before.

To do this, the GPIO port enabler in the reset clock control register (RCC) that controls over the peripheral clock of a certain port of the registers must be applied. The GPIO port enabler have options like enable or disable a certain port of choice in the provided STM device so it can measure which registers that can be used by the software to read or writes the attached pins of the board.

And in this case, the RCC register to activate the peripheral clock which will be used for the LED output is the GPIO port B. And for the input, it is the GPIO port A.

In this Figure 2. 2. 1 below, the position of GPIO port A can be seen as GPIOA EN. And the position of GPIO port B can be seen as GPIOB EN

|     |     |             |         |     |     |     |       |                           |            |                          |             |            |            |            |                           |
|-----|-----|-------------|---------|-----|-----|-----|-------|---------------------------|------------|--------------------------|-------------|------------|------------|------------|---------------------------|
| 31  | 30  | 29          | 28      | 27  | 26  | 25  | 24    | 23                        | 22         | 21                       | 20          | 19         | 18         | 17         | 16                        |
| Res | Res | ADC34<br>EN | ADC12EN | Res | Res | Res | TSCEN | IOPG<br>EN <sup>(1)</sup> | IOPF<br>EN | IOPE<br>EN               | IOPD<br>EN  | IOPC<br>EN | IOPB<br>EN | IOPA<br>EN | IOPH<br>EN <sup>(1)</sup> |
|     |     | rW          | rW      |     |     |     | rW    | rW                        | rW         | rW                       | rW          | rW         | rW         | rW         | rW                        |
| 15  | 14  | 13          | 12      | 11  | 10  | 9   | 8     | 7                         | 6          | 5                        | 4           | 3          | 2          | 1          | 0                         |
| Res | Res | Res         | Res     | Res | Res | Res | Res   | Res                       | CRC<br>EN  | FMC<br>EN <sup>(1)</sup> | FLITF<br>EN | Res        | SRAM<br>EN | DMA2<br>EN | DMA1<br>EN                |
|     |     |             |         |     |     |     |       |                           | rW         | rW                       | rW          |            | rW         | rW         | rW                        |

1. Only on STM32F303xDxE.

Figure 2. 2. 1 RCC GPIO register enabler map.

After the corresponding GPIO port is enabled, the pin inside that port can be set up manually using some other bit manipulations, depending on the pin number that is going to be used.

The setups are mostly about the mode of the pin (MODER), the speed response of the pin (SPEEDR), and the resistor setup (PUPDR).

This setup can be seen from this Figure 2. 2. 2 below.

To be set in an output mode, the MODER of the chosen pin must be masked to the general-purpose output mode value.

And since the output pin that will be used in the project is PB1 & PB2, MODER\_1 and MODER\_2 in GPIO B shall be set to general output purpose mode.

On the other hand, the PA8 & PA9 which are needed for the GPIO input shall be set to the reset state.

|              |    |              |    |              |    |              |    |              |    |              |    |             |    |             |    |
|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|-------------|----|-------------|----|
| 31           | 30 | 29           | 28 | 27           | 26 | 25           | 24 | 23           | 22 | 21           | 20 | 19          | 18 | 17          | 16 |
| MODER15[1:0] |    | MODER14[1:0] |    | MODER13[1:0] |    | MODER12[1:0] |    | MODER11[1:0] |    | MODER10[1:0] |    | MODER9[1:0] |    | MODER8[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |
| 15           | 14 | 13           | 12 | 11           | 10 | 9            | 8  | 7            | 6  | 5            | 4  | 3           | 2  | 1           | 0  |
| MODER7[1:0]  |    | MODER6[1:0]  |    | MODER5[1:0]  |    | MODER4[1:0]  |    | MODER3[1:0]  |    | MODER2[1:0]  |    | MODER1[1:0] |    | MODER0[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |

Bits 2y+1:2y **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figure 2. 2. 2 GPIO mode register map.

For the output pins, after setting up the mode register, the type of the output must be determined. In this case, the OTYPER register of PB1 in the board is being set to the Output push-pull mode, which means writing the register 1 of this Figure 2. 2. 3 below to 0b0 into its reset state, to control its high or low state digitally when toggling the PB1 output later.

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 31   | 30   | 29   | 28   | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
| 15   | 14   | 13   | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| OT15 | OT14 | OT13 | OT12 | OT11 | OT10 | OT9  | OT8  | OT7  | OT6  | OT5  | OT4  | OT3  | OT2  | OT1  | OT0  |
| rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Figure 2. 2. 3 GPIO output type register map.

After setting up the output type, the speed of the pin can be determined by toggling the OSPEEDR register value in this board. Based on the Figure 2. 2. 4 below, the speed of the LED is set to 0b11 which means the high speed.

|                    |    |                    |    |                    |    |                    |    |                    |    |                    |    |                   |    |                   |    |
|--------------------|----|--------------------|----|--------------------|----|--------------------|----|--------------------|----|--------------------|----|-------------------|----|-------------------|----|
| 31                 | 30 | 29                 | 28 | 27                 | 26 | 25                 | 24 | 23                 | 22 | 21                 | 20 | 19                | 18 | 17                | 16 |
| OSPEEDR15<br>[1:0] |    | OSPEEDR14<br>[1:0] |    | OSPEEDR13<br>[1:0] |    | OSPEEDR12<br>[1:0] |    | OSPEEDR11<br>[1:0] |    | OSPEEDR10<br>[1:0] |    | OSPEEDR9<br>[1:0] |    | OSPEEDR8<br>[1:0] |    |
| rw                 | rw | rw                 | rw | rw                 | rw | rw                 | rw | rw                 | rw | rw                 | rw | rw                | rw | rw                | rw |
| 15                 | 14 | 13                 | 12 | 11                 | 10 | 9                  | 8  | 7                  | 6  | 5                  | 4  | 3                 | 2  | 1                 | 0  |
| OSPEEDR7<br>[1:0]  |    | OSPEEDR6<br>[1:0]  |    | OSPEEDR5<br>[1:0]  |    | OSPEEDR4<br>[1:0]  |    | OSPEEDR3<br>[1:0]  |    | OSPEEDR2<br>[1:0]  |    | OSPEEDR1<br>[1:0] |    | OSPEEDR0<br>[1:0] |    |
| rw                 | rw | rw                 | rw | rw                 | rw | rw                 | rw | rw                 | rw | rw                 | rw | rw                | rw | rw                | rw |

Bits  $2y+1:2y$  **OSPEEDRy[1:0]**: Port x configuration bits ( $y = 0..15$ )

These bits are written by software to configure the I/O output speed.

x0: Low speed

01: Medium speed

11: High speed

*Note: Refer to the device datasheet for the frequency specifications and the power supply and load conditions for each speed.*

Figure 2. 2. 4 GPIO speed register map.

And the last one is resetting the PUPD\_1 & PUPD\_2 register to 0b00 since both output pins do not use the pull-up/pull-down resistor.

After these setups are finished, the PB1 & PB2 are now ready to be toggled by the STM system to be HIGH or LOW.

And on the other hand, the input triggers only need the PUPD and OSPEEDR registers. On the PUPD registers, both PA8 & PA9 must be set to pull-down. And the speed of both registers can be equal to the output pins.

### 2.3. Setting up the GPIO EXTI

Finished on the GPIO setup, now the EXTI setup can be created, following to the instructions on the Preparation section above.

To be known, because the inputs are using pin 8 & pin 9 in port A, the system EXTI line for these pins are the combined EXTI\_5 to EXTI\_9.

After everything is setup, the main while loop of the program can be tested by adding a very huge delay such as shown in this Figure 2. 3. 1 below.

```

110
111     /* Infinite loop */
112     /* USER CODE BEGIN WHILE */
113     while (1) {
114         /* USER CODE END WHILE */
115         HAL_Delay(1000000000);
116         /* USER CODE BEGIN 3 */
117     }
118     /* USER CODE END 3 */
119 }
120

```

Figure 2. 3. 1 A large delay to test out the interrupt handler.

This huge delay can be implemented to test if the interrupt configuration is successful or not after the EXTI handler function is being placed in the program.

Once the program is running properly, the result must likely be similar to the manual testing result, written in Button toggle result section below this document.

The source code of the corresponding program can be accessed via the git link, written in References section of this document.

## 2.4. Experiment (Polling Interrupt)

To make this assignment more fun and interactive, an experiment such as creating a row of sweep LEDs, combined it with the toggling LED-pushbutton assignment can be created.

The concept of this experiment is to switch the sweep mode of the row of the connected LEDs to the STM32 board by using the button.

There are three modes of the sweep LEDs. The first one is the single LED sweep mode, the second one is the multiple LEDs sweep mode, and the last one is the off or reset mode.

To do this, the system needs four more extra LEDs with 220  $\Omega$ ,  $\pm 5\%$  resistors, also a press button with a 1000  $\Omega$ ,  $\pm 5\%$  resistor. The wiring method for this experiment is presented in this Figure 2. 4. 1 below.

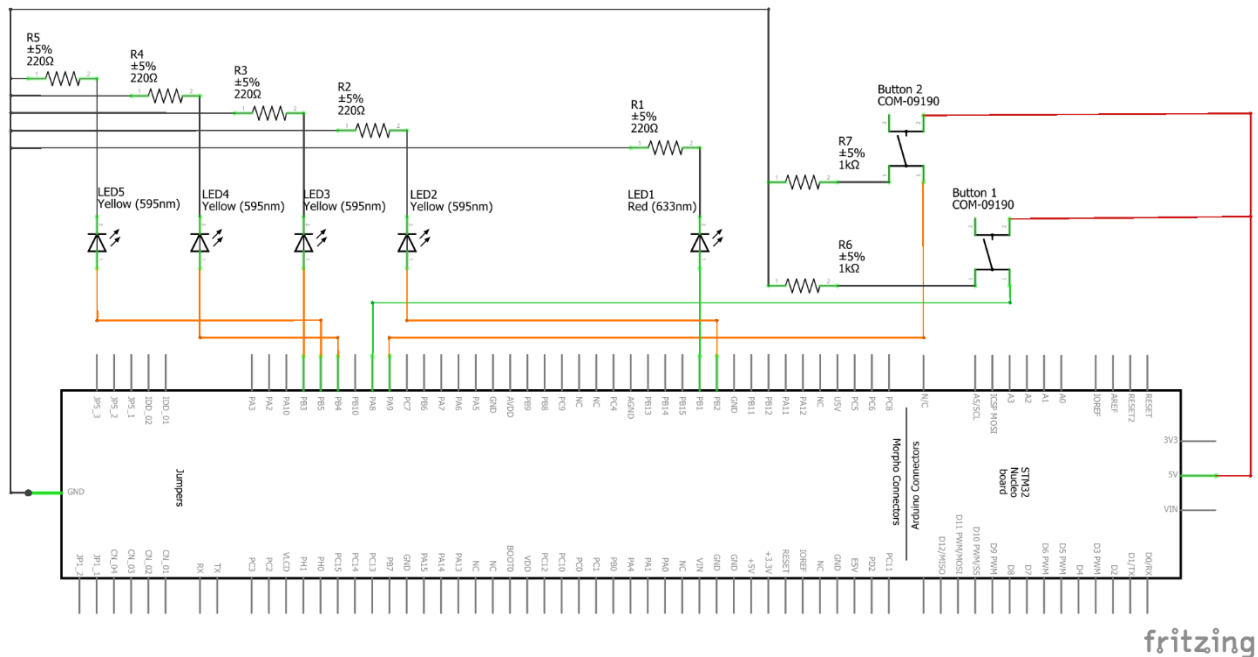


Figure 2. 4. 1 Extended wiring for polling interrupt experiment.

The program should run the sweep states in polling, following to the design concept which is presented in this Polling Interrupts section below.

And the polling must be changed, due to the command received within the interrupt handler.

To enable the extra LED outputs, the previous setup can be extended, following the necessary pins needed which is depicted in Figure 2. 4. 1 above. As for example, by using the “|” (OR) pipe symbol in the setup of the GPIO B.

And this method also applied to the EXTI register configuration.

This Figure 2. 4. 2 below is an example of the extended GPIO B setup for the output LEDs.

```

void gpio_b_init(void) {
    //resets all the LEDs' pin MODE.
    GPIOB->MODER &= ~((0x3UL << GPIO_MODER_MODER1_Pos)
        | (0x3UL << GPIO_MODER_MODER2_Pos)
        | (0x3UL << GPIO_MODER_MODER3_Pos)
        | (0x3UL << GPIO_MODER_MODER4_Pos)
        | (0x3UL << GPIO_MODER_MODER5_Pos));
    //resets all LEDs' pin OTYPE to 0 as push-pull.
    GPIOB->OTYPER &= ~(GPIO_OTYPER_OT_1 | GPIO_OTYPER_OT_2
        | GPIO_OTYPER_OT_3 | GPIO_OTYPER_OT_4 | GPIO_OTYPER_OT_5);
    //resets all LEDs' pin SPEED.
    GPIOB->OSPEEDR &= ~((0x3UL << GPIO_OSPEEDER_OSPEEDR1_Pos)
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR2_Pos)
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR3_Pos)
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR4_Pos)
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR5_Pos));

    //sets the LEDs' pin MODE to 01 as an output.
    GPIOB->MODER |= ((0x1UL << GPIO_MODER_MODER1_Pos)
        | (0x1UL << GPIO_MODER_MODER2_Pos)
        | (0x1UL << GPIO_MODER_MODER3_Pos)
        | (0x1UL << GPIO_MODER_MODER4_Pos)
        | (0x1UL << GPIO_MODER_MODER5_Pos));
    //sets all LEDs' pin SPEED to medium.
    GPIOB->OSPEEDR |= ((0x1UL << GPIO_OSPEEDER_OSPEEDR1_Pos)
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR2_Pos)
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR3_Pos)
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR4_Pos)
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR5_Pos));
    //resets all LEDs' PUPD register.
    GPIOB->PUPDR &= ~((0x3UL << GPIO_PUPDR_PUPDR1_Pos)
        | (0x3UL << GPIO_PUPDR_PUPDR2_Pos)
        | (0x3UL << GPIO_PUPDR_PUPDR3_Pos)
        | (0x3UL << GPIO_PUPDR_PUPDR4_Pos)
        | (0x3UL << GPIO_PUPDR_PUPDR5_Pos));
}

```

Figure 2. 4. 2 Extended output GPIO.

After all the necessary pins are being enabled, now the sweeping program can be written.

The idea of the sweep is to make a row of LEDs to be on and off, queued by their time and position in a loop.

To do the sweep switch, another debouncing method for the second input button must be created.

After that, a switch method to change the sweep states from each press trigger from the button can be added to the program.

This switch method can be seen in Figure 2. 4. 3 as follows.

```
126  /* Infinite loop */
127  /* USER CODE BEGIN WHILE */
128  while (1) {
129      /* USER CODE END WHILE */
130      if (sweep_state == OFF) {
131          all_sweep_off();
132      } else if (sweep_state == SINGLE) {
133          sweep_a(&sweep_a_counter, &sweep_a_to_left);
134      } else if (sweep_state == MULTI) {
135          sweep_b(&sweep_b_counter, &sweep_b_adder, &sweep_b_reverse,
136                &sweep_b_to_left);
137      }
138      /* USER CODE BEGIN 3 */
139  }
140  /* USER CODE END 3 */
141 }
```

Figure 2. 4. 3 Polling method to test the interrupt experiment.

As per-function shown in this Figure 2. 4. 3 above, sweep\_a() is the function to executes the single sweep mode, the sweep\_b() is the function to executes the multiple sweep mode, and the all\_sweep\_off() is the function to turn off all of the sweep LEDs.

These sweeps are based on the enumeration states created in the header file such as this Figure 2. 4. 4 below.

```
39 typedef enum
40 {
41     OFF,
42     SINGLE,
43     MULTI
44 }MySweepState;
45
```

Figure 2. 4. 4 Enumeration of sweep states.

The result of the manual testing of this program can be seen in the Experiment result section below this document. And the source file can be accessed via a git link in the References section of this document.

### 3. Design Concept

This section of the report contains the logic state diagrams of each experiment to proof the concept of interrupts.

#### 3.1. Basic Interrupts

This proof of concept is basically using a simple switch method to do the LED toggles based on the timer tick of the button press and the current state of the LEDs.

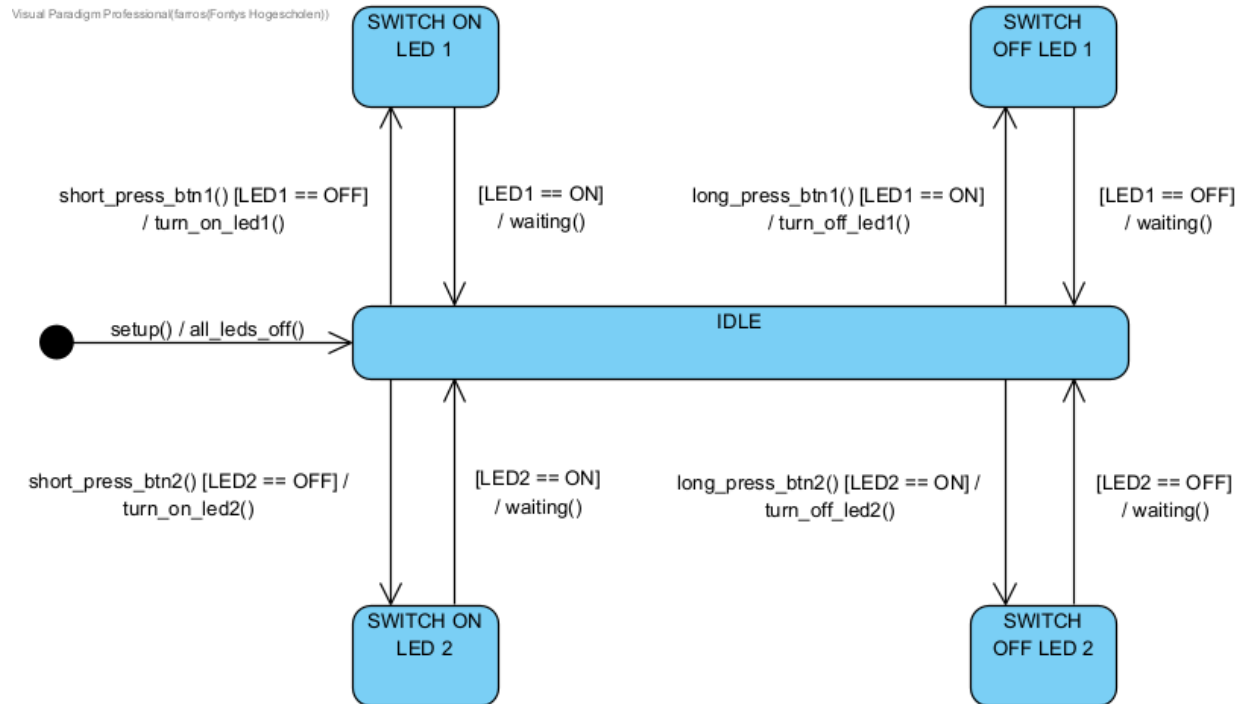


Figure 3. 1. 1 Basic interrupt state machine.

In this Figure 3. 1. 1 above, the system starts by doing its setup and turn off the two LEDs before it is going to the idle state.

This idle state actually a waiting state which can be extended to do another polling program under its sub state machine.

If an interrupt button happens within the short delay, the system will turn on the LED and go back to its original polling state. And if a long delay interrupt happens, the system will turn off the LED and go back to its original polling state.

### 3.2. Polling Interrupts

This proof of concept is an extended version of the previous one. The sweep change is triggered by the button input interrupt while the sweep state itself is a polling loop program.

In this Figure 3. 2. 1 below, the system can still do the sweep while the single LED is being switched on and off.

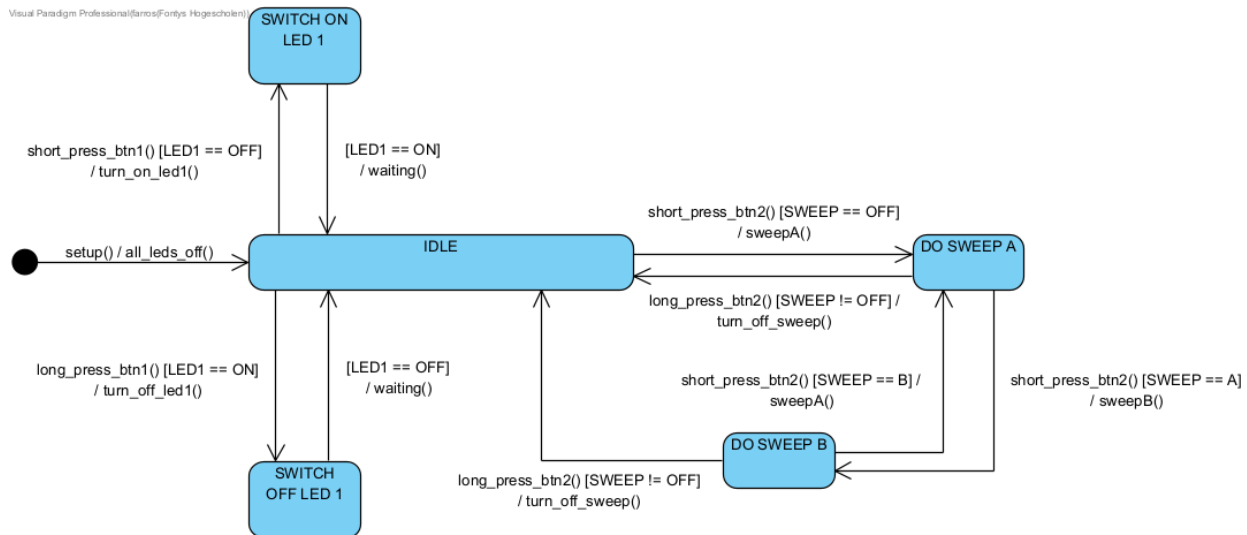


Figure 3. 2. 1 Interrupt experiment state machine.

From that Figure 3. 2. 1 above, the system starts to IDLE state to wait triggers, then switch to SWEEP A if the second button is pressed shortly. At the same time, while the system is on the IDLE mode, if the first button is pressed shortly, the system will switch on the single LED.

SWEEP A can be changed to SWEEP B and go back to SWEEP A again if the second button is pressed shortly. And if each button is being pressed longer the current state should make the corresponding LEDs to off.

## 4. Results

After doing these practices, some simulations and manual testing can be made by using a logic analyzer or serial monitor. In this case, the logic manual testing used in this practicum is using a logic analyzer device named as The Scana, IKA Logic software where the device version is SQ50.

The result of each testing on each LED device product can be seen below here, as follows, in each section of the practicum.

And the serial monitor testing in this project is using the UART transmit protocol from the STM32 Nucleo board to a serial monitoring application named as the Termite.

## 4.1. Button toggle result

At the beginning of the program after it is being ran, both the LEDs and the button states should be low as expected. And when the short press on a button is happening, the corresponding LED should switch to high as what is shown in this Figure 4. 1. 1 below.

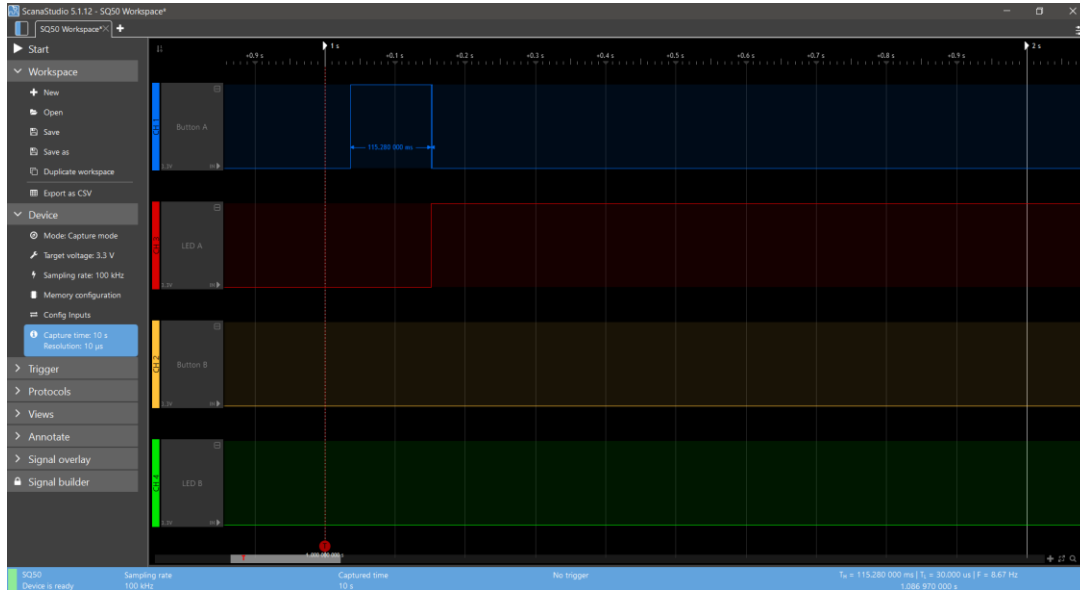


Figure 4. 1. 1 Logic analyzer capture on button click interrupt.

After that, if the same button is being pressed again but on the longer delay, the corresponding LED should be switched to LOW, as like what is being shown in this Figure 4. 1. 2 below.

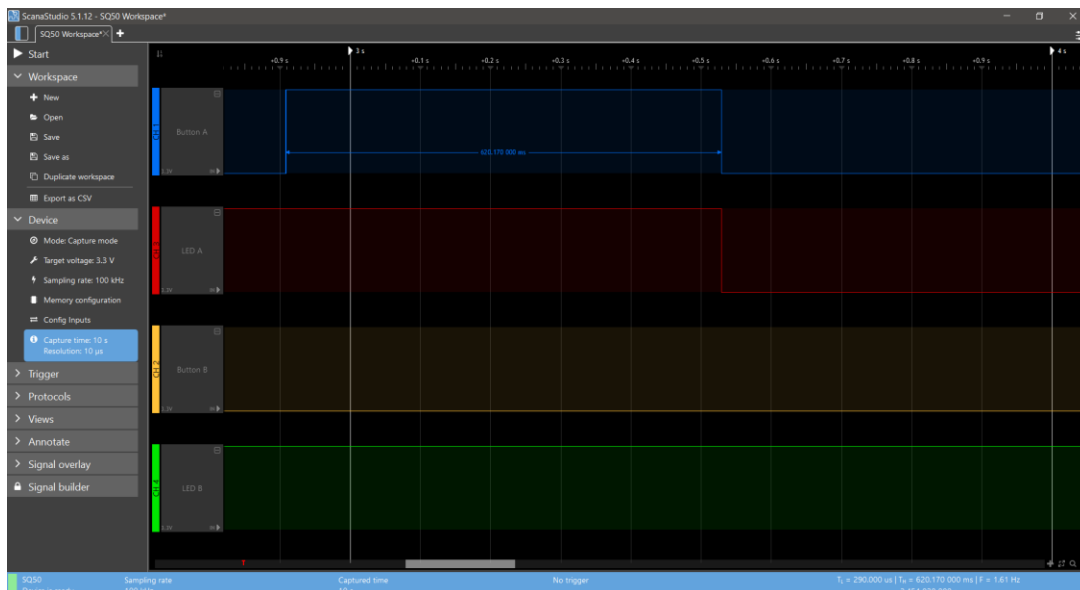


Figure 4. 1. 2 Logic analyzer capture on switch of LED A.

With the external interrupt method, button A and B will change the state of LED A and B without waiting for each of their turn in a polling loop such as shown in this Figure 4. 1. 3 below.

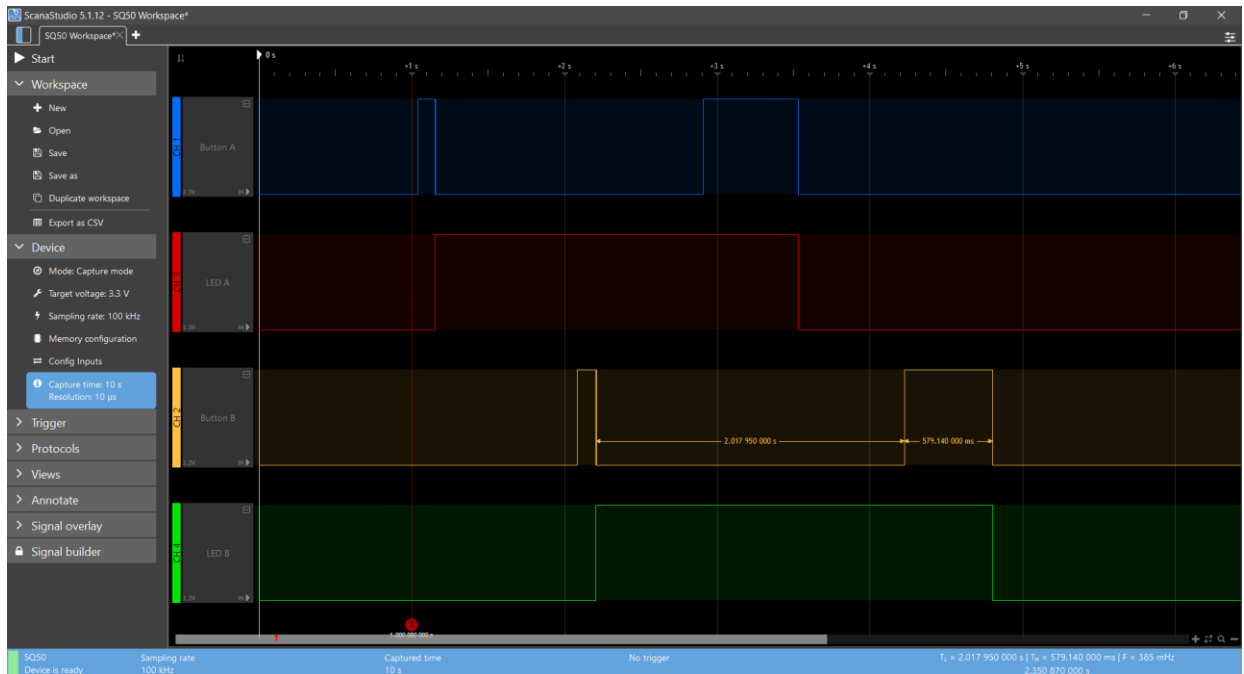


Figure 4. 1. 3 Logic analyzer capture on interrupt button.

This proof-of-concept result can also be watched in the serial monitor after using the UART communication, every time after the click happened, such as shown in this Figure 4. 1. 4 below.

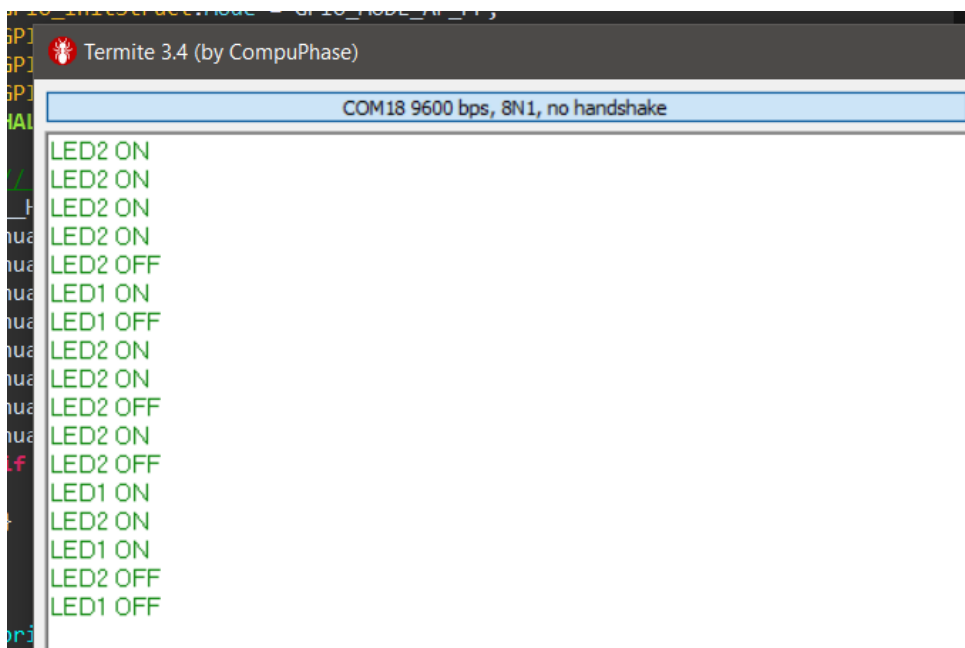


Figure 4. 1. 4 Serial print result from the proof of concept manual testing.

## 4.2. Experiment result

The result of the experiment is a little bit different than the previous practice because each LED is switched on and off periodically in a certain queue and timing.

In this Figure 4. 2. 1 below, it is visible that in the sweep a, each LED is on and off by its order to forward and backward.

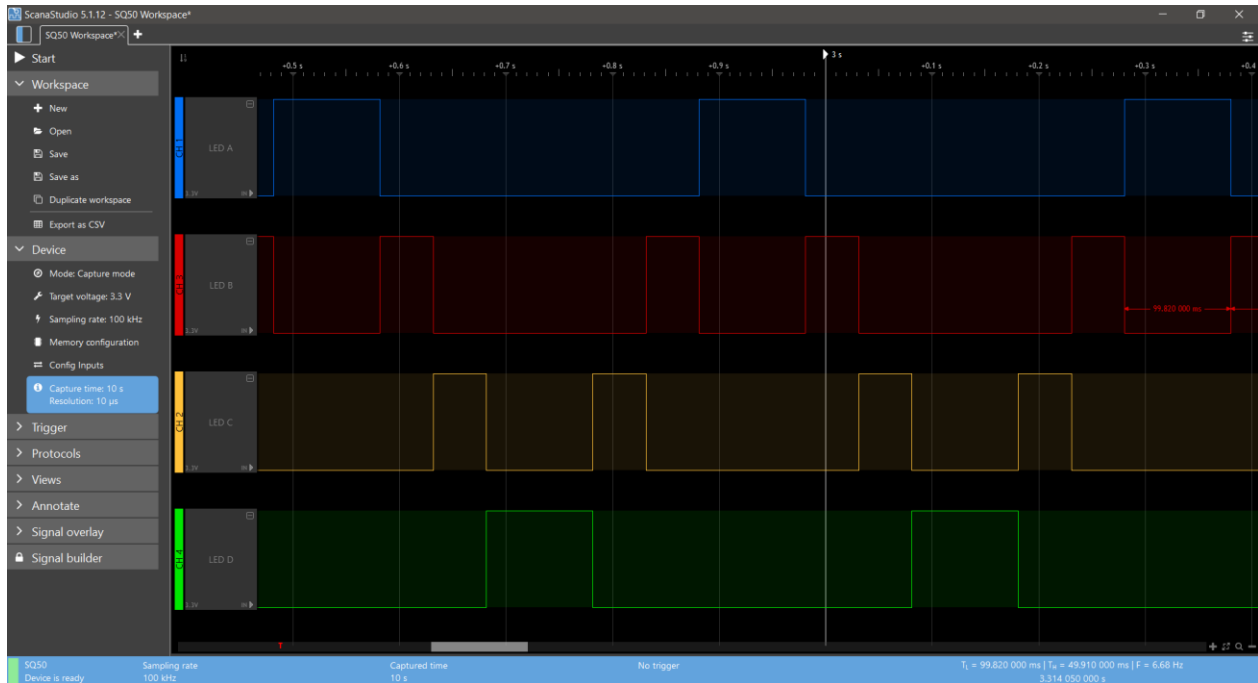


Figure 4. 2. 1 Sweep A polling result.

And this also happens on the sweep b, where all LEDs turned on, one by one, by their order, before they go off, one by one, by their order due to the polling counter. This is visible in this Figure 4. 2. 2 below.

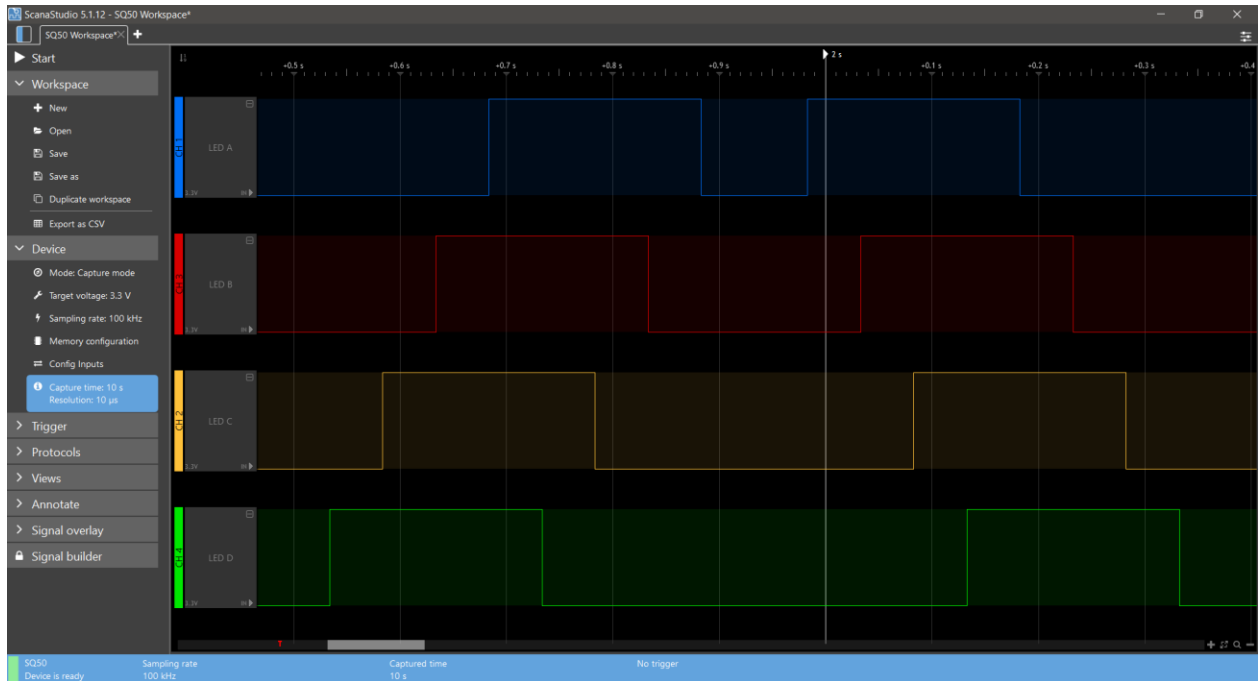


Figure 4. 2. 2 Logic analyzer result on sweep b.

But when the interrupt button read happens, the system will switch state immediately such as shown in this Figure 4. 2. 3 below.

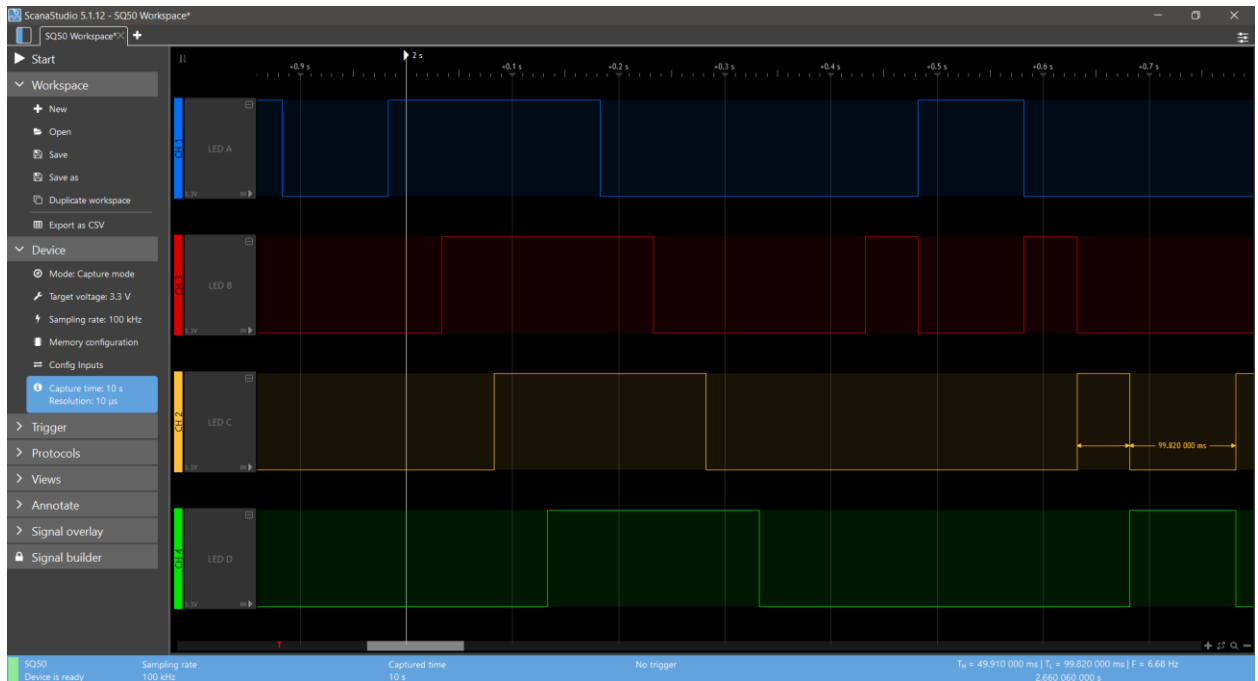


Figure 4. 2. 3 Logic analyzer capture on the sweep switch by interrupts.

This proof of concepts can also be visible in the serial monitor such is shown in this Figure 4. 2. 4 below if the UART communication is configured.

```
@note this function is a custom GPIO UART initialization.
```

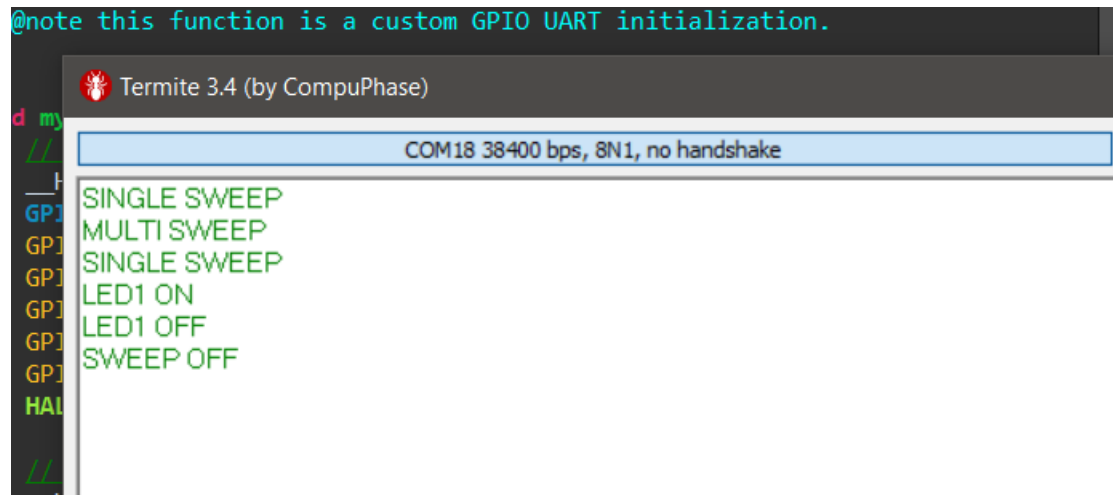


Figure 4. 2. 4 Manual testing result on serial print.

## 5. Conclusion

Based on the results, after doing this practicum, it is becoming so handy to understand the purpose of interrupt in the STM32 board that it could avoid busy waiting polling method such as shown in the extra experiment.

The system is becoming more modular than just doing it with polling, delay, and just a basic GPIO setup.

With this modular capability, the system could do multiple tasks at once without waiting for each processed programs to be finished first which results a good power management of the system.

And the system itself become much more efficient than only doing the basic GPIO setup with delay in the polling main loop.

## 6. Recommendation

Further experiments can be added by using this interrupt project as its basis reference in the future. Most embedded system project can be optimized by using this interrupt method. Moreover, a project that must do a multi-tasking program such an RTOS.

And doing an interrupt handling must be explored more carefully, knowing that not all pins in the GPIO port have their own specific EXTI line such as EXTI\_5 until EXTI\_9.

## 7. References

Along this practicum, such documents and references are being used.

- DM00043574-ReferenceManual\_STM32F303RE([www.st.com](http://www.st.com)), January 2017
- UM1724 USER manual STM32 Nucleo-64 boards (MB1136) ([www.st.com](http://www.st.com)), April 2019
- Arduino Pull-up Pull-down Resistor ([https://arduinogetstarted.com/fag/arduino-pull-up-pull-down-resistor?utm\\_content=cmp-true](https://arduinogetstarted.com/fag/arduino-pull-up-pull-down-resistor?utm_content=cmp-true))
- Resistor Color Code Calculator (<https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-resistor-color-code>)

Git link to access the source file of the assignment:

- Interrupts:  
<https://git.fhict.nl/l424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/2-interrupts>